Problem Set 1

Due date: March 9, 2026 before class

倪伟丰 2024110089

1. (Exercise 1.5 from the textbook) The Stable Matching Problem, as discussed in the text, assumes that all students and hospitals have a fully ordered list of preferences. In this problem we will consider a version of the problem in which students and hospitals can be indifferent between certain options. As before we have a set S of n students and a set H of n hospitals. Assume each student and each hospital ranks the members of the other type, but now we allow ties in the ranking. For example (with $n = 4$), a hospital could say that s_1 is ranked in first place; second place is a tie between s_2 and s_3 (it has no preference between them); and s_4 is in last place. We will say that h prefers s to s' if s is ranked higher than s' on its preference list (they are not tied). With indifferences in the rankings, there could be two natural notions for stability. And for each, we can ask about the existence of stable matchings, as follows.

   (a) A strong instability in a perfect matching M consists of a student s and a hospital h, such that each of s and h prefers the other to their partner in S. Does there always exist a perfect matching with no strong instability? Either give an example of a set of students and hospitals with preference lists for which every perfect matching has a strong instability; or give an algorithm that is guaranteed to find a perfect matching with no strong instability.

   (b) A weak instability in a perfect matching M consists of a student s and a hospital h, such that their partners in M are h' and s', respectively, and one of the following holds:

   - s prefers h to h', and h either prefers s to s' or is indifferent between these two choices; or
   - h prefers s to s', and s either prefers h to h' or is indifferent between these two choices.

   In other words, the pairing between s and h is either preferred by both, or preferred by one while the other is indifferent. Does there always exist a perfect matching with no weak instability? Either give an example of a set of students and hospitals with preferences lists for which every perfect matching has a weak instability, or give an algorithm that is guaranteed to find a perfect matching with no weak instability.

Solution to (a). There always exist a perfect matching with no strong instability.

The algorithm is a modification of the Gale-Shapley algorithm on our textbook. It is guaranteed to find a perfect matching with no strong instability. The proof of correctness is similar to the proof of the original Gale-Shapley algorithm, with the added consideration of ties in preferences, if a student meets the condition of a tie of two hospitals, the student can propose to either hospital. And the key idea is that once a student proposes to a hospital, that hospital will never reject that

Algorithm 1 G-S Algorithm (for Strong Instability)

Initially all $s \in S$ and $h \in H$ are free
while there exists a student s who is free and has not proposed to every hospital **do**
  Choose such a student s
  Let h be the highest-ranked hospital (not dominated by any unproposed hospital) in s's preference list to whom s has not yet proposed
  if h is free **then**
    (s, h) become engaged
  else $\{h$ is currently matched to $s'\}$
    if h prefers s' to s or is indifferent between s and s' **then**
      s remains free
    else $\{h$ prefers s to $s'\}$
      (s, h) become engaged
      s' becomes free
    end if
  end if
end while
Return the set M of matched pairs

student in favor of another student who is ranked lower or tied with the current student. This ensures that there are no strong instabilities in the final matching.

As for the proof of termination amd perfect matching, it is the same as the original Gale-Shapley algorithm on our textbook.

The proof of no strong instability. Suppose for contradiction that there is a strong instability in the final matching, which means there exists a student s and a hospital h that s prefers h to their current partner, and h prefers s to their current partner. Has s ever proposed to h during the algorithm? If not, the according to the algorithm, it must be that s does not strictly prefer h to its final match, which contradicts the assumption that s prefers h to their current partner. If s has proposed to h, then h must have rejected s and chosen another student s'. This means that h prefers s' to s or is indifferent between s and s', which contradicts the assumption that h prefers s to their current partner as well. □

□

Solution to (b). There does not always exist a perfect matching with no weak instability. Suppose we have a set of two student, $\{s, s'\}$, and a set of two hospital, $\{h, h'\}$. The preference lists are as follows:

$$\begin{gathered} s \text{ is indifferent between } h \text{ and } h'; \\ s' \text{ is indifferent between } h \text{ and } h'; \\ h \text{ prefers } s \text{ to } s'; \\ h' \text{ prefers } s \text{ to } s'. \end{gathered}$$

In this case, there are two perfect matchings: $M_1 = \{(s, h), (s', h')\}$ and $M_2 = \{(s, h'), (s', h)\}$. Both of these matchings have a weak instability. In M_1, the pair (s, h') is a weak instability because h' prefers s to s' and s is indifferent between the two hospitals. In M_2, the pair (s, h) is a weak instability because h prefers s to s' and s is indifferent between the two hospitals. Therefore, in this example, every perfect matching has a weak instability. □

2. (Exercise 1.6 from the textbook) Peripatetic Shipping Lines Inc., is a shipping company that owns n ships and provides service to n ports. Each of its ships has a schedule that says, for each day of the month, which of the ports it's currrently visiting, or whether it's out at sea. (You can assume the "month" here has m days, for some $m > n$.) Each ship visits each port for exactly one day during the month. For safety reasons, PSL Inc. has the following strict requirement:

   (†) No two ships can be in the same port on the same day.

   The company wants to perform maintenance on all the ships this month, via the following scheme. They want to truncate each ship's schedule: for each ship S_i, there will be some day when it arrives in its scheduled port and simply remains there for the rest of the month (for maintenance). This means that S_i will not visit the remaining ports on its schedule (if any) that month, but this is okay. So the truncation of S_i's schedule will simply consist of its original schedule up to a certain specified day on which it is in a port P; the remainder of the truncated schedule simply has it remain in port P.

   Now the company's question to you is the following: Given the schedule for each ship, find a truncation of each so that condition continues to hold: no two ships are ever in the same port on the same day.

   Show that such a set of truncations can always be found, and give an algorithm to find them.

   Example. Suppose we have two ships and two ports, and the "month" has four days. Suppose the first ship's schedule is

   port P_1; at sea; port P_2; at sea

   and the second ship's schedule is

   at sea; port P_1; at sea; port P_2

   Then the (*only*) way to choose truncations would be to have the first ship remain in port P_2 starting on day 3, and have the second ship remain in port P_1 starting on day 2.

   Solution. We can solve this problem using a greedy algorithm, referencing the G-S algorithm and Interval Scheduling problem in Chapter 1.

   First, we regulate the symbol of the problem:

S is the set of ships,
P is the set of ports,
D is the set of days,
$L(P_i)$ means the latest day that any ship visits port P_i,
$S(P_i)$ means the ship that visits port P_i on day $L(P_i)$.

Before the algorithm, it's easy to verify that the prerequisite of the truncation of a ship S_i at port P_j is that S_i must be the last one in un-truncated ships who visits P_j in the month, which means $S_i = S(P_j)$.

What's more, we can also verify that each ship visits each port for exactly one day during the month.

Algorithm 2 Greedy Algorithm for Truncation

Initially all ships' schedules are un-truncated, and all ports are un-truncated.
For all ports P_i, calculate $L(P_i)$ and $S(P_i)$.
while there exists an un-truncated port **do**
  Choose an un-truncated port P_j with the minimum $L(P_j)$.
  Truncate ship $S(P_j)$ starting from day $L(P_j)$ and keep it at port P_j.
  Remove ship $S(P_j)$ and port P_j from further consideration.
  Recompute $L(P_i)$ and $S(P_i)$ for all remaining ports.
end while

Correctness. At any stage of the algorithm, let P^* be a remaining port with minimum $L(P)$, and let $s^* = S(P^*)$. We can verify that:

Claim. In every feasible truncation of the current subproblem, ship s^* must be truncated at port P^* on day $L(P^*)$.

Proof of claim. Suppose for contradiction that there is a conflict in the schedule, which means there exists two ships S_i and S_k that are in the same port P_j on the same day d, and day d is the earliest day on which such a conflict occurs. Let's suppose that S_i is the ship that was truncated at port P_j on t_i before day d, which means that $S_i = S(P_j)$ and $t_i = L(P_j)$. We show the case by S_i, and the case by S_k is similar. What's more the conflict can only happen when a ship is truncated at a port, since the original schedule of each ship is conflict-free. Why ship S_k can visit P_j on day d? If ship S_k visits P_j by the original schedule, we can get that $t_i < L(P_j)$, because there is a ship visit port P_j after day t_i, which contradicts the fact that S_i is the last one in un-truncated ships who visits P_j in the month. What's more, if one port is truncated, the ship that visits this port on the day of truncation will never leave this port, the specific ship and specific port of truncation is determined and removed from following decision by the algorithm. Thus there is no conflict in the schedule. □

Induction. If there is only one remaining port, then there is only one remaining ship, and truncating that ship at that port is clearly feasible. Assume the algorithm is correct whenever there

are $k-1$ remaining ports, and consider a stage with k remaining ports. By the claim above, every feasible solution must truncate s^* at P^* on day $L(P^*)$. So the greedy choice is forced. Remove s^* and P^*. The remaining instance is of the same form: every remaining ship still visits every remaining port exactly once, and the original schedules among the remaining ships are still conflict-free. By the induction hypothesis, the algorithm finds feasible truncations for this smaller instance. We only need to check that adding back the choice of s^* at P^* creates no conflict. Before day $L(P^*)$, ship s^* follows its original schedule, so there is no conflict. From day $L(P^*)$ onward, ship s^* stays at P^*. But no remaining ship ever visits P^* after day $L(P^*)$, by definition of $L(P^*)$. Hence s^* cannot conflict with any remaining ship. Therefore the combined truncation is feasible. This proves that the greedy algorithm always produces a feasible set of truncations. □

Running time and Termination. In each iteration, there are at most n remaining ports. For each remaining port we may scan the schedules of the remaining ships over the m days to determine $L(P)$ and $S(P)$, which takes $O(mn)$ time per iteration. Since there are n iterations, the total running time is $O(mn^2)$ and the algorithm will end in $O(mn^2)$ time. □

□

(Exercise 1.8 from the textbook) For this problem, we will explore the issue of truthfulness in the Stable Matching Problem and specifically in the Gale-Shapley algorithm. The basic question is: Can a student or a hospital end up better off by lying about their preferences? More concretely, we suppose each participant has a true preference order. Now consider a hospital h. Suppose h prefers student s to s', but both s and s' are low on its list of preferences. Can it be the case that by switching the order of s and s' on its list of preferences (i.e., by falsely claiming that it prefers s' to s) and running the algorithm with this false preference list, h will end up with a student s'' that it truly prefers to both s and s'? (We can ask the same question for students, but will focus on the case of hospitals for purpose of this question.)

Resolve this question by doing one of the following two things:

(a) Give a proof that, for any set of preference lists, switching the order of a pair on the list cannot improve a hospital's partner in the Gale-Shapley algorithm; or

(b) Give an example of a set of preference lists for which there is a switch that would improve the partner of a hospital who switched preferences.

Solution. If the hospital is the proposee, then there exists an example of a set of preference lists for which there is a switch that would improve the partner of a hospital who switched preferences; If the hospital is the proposor, then switching the order of a pair on the list cannot improve a hospital's partner in the Gale-Shapley algorithm.

The hospital is the proposee. Consider the following true preference lists:

- Hospital h_1: $s_2 > s_1 > s_3$
- Hospital h_2: $s_1 > s_2 > s_3$

- Hospital h_3: $s_1 > s_2 > s_3$
- Student s_1: $h_1 > h_2 > h_3$
- Student s_2: $h_2 > h_1 > h_3$
- Student s_3: $h_1 > h_2 > h_3$

In this case, the Gale-Shapley algorithm will produce the following matching:

h_1 is matched with s_1;
h_2 is matched with s_2;
h_3 is matched with s_3.

However, if hospital h_1 switches the order of s_1 and s_3 on its preference list and forms a fake preference list as follows:

- Hospital h_1: $s_2 > s_3 > s_1$
  …(other hospitals and students preferences remain)

Then the Gale-Shapley algorithm will produce the following matching:

h_1 is matched with s_2;
h_2 is matched with s_1;
h_3 is matched with s_3.

We find that by switching the order of s_1 and s_3 on its preference list, hospital h_1 ends up with student s_2, who is truly preferred to both s_1 and s_3. Therefore, this example demonstrates that a hospital can benefit from lying about its preferences in the G-S algorithm. □

The hospital is the proposer. Assume that hospitals are the proposers in the G-S algorithm. And then switching the order of a pair on the list cannot improve a hospital's match in G-S algorithm. Following is the proof.

Let hospital h have true preference list in which h prefers student s to s', and both s and s' are low on its list of preferences, Then suppose h reports a false preference list obtained by swapping the positions of s and s'.

We claim that this switch cannot make h end up with a student s'' satisfying:

$$s'' \succ s \qquad \text{and} \qquad s'' \succ s'$$

Suppose for contradiction that, under the false preference list, hospital h is matched with some student s'' whom it truly prefers to both s and s'.

h is the proposer, who makes proposals according to his preference list. Because s'' is truly preferred to both s and s', and the false list differs from the true list only in the switched order of s and s', the student s'' still appears before both s and s' in the false list. Hence, in both times of the G-S algorithm (truthful and false), hospital h proposes to the same sequence of students before the more preferred between s and s' on the preference list no matter true or false.

Let's consider the false preference list, hospital h is ultimately matched with s''. Therefore, after proposing to s'', hospital h never proceeds to any student ranked below s''; in particular, it never reaches either s or s'. Then we can find that the part of the preference list where the truthful and false one differs is never used and never influences the result.

Thus the entire execution of hospital h is identical under the truthful and false list: before reaching s or s', the proposal sequence is the same, and in fact neither s nor s' is ever proposed to. Therefore h must also be matched with s'' under truthful reporting.

This contradicts the assumption that swapping s and s' can cause h to obtain such a student s''. Therefore, swapping the order of s and s' cannot make a proposing hospital obtain a student whom it truly prefers to both s and s'. □